IIITDM KANCHEEPURAM

Department of Computer Science and Engineering

MergeMind

2048 Number-Merging Puzzle

Object-Oriented Programming

Project Report

Submitted by: CS25B1002 – R Himsesha

CS25B1019 – B Nandha Gopal

CS25B1024 – G Preetham Chowday

CS25B1019 – P Rohith

Course Code: 25CS2001

Academic Year: 2026–2027

Submitted in partial fulfilment of the course requirements

October 2026

Abstract

MergeMind is a terminal-based implementation of the 2048 puzzle in C++. The player slides numbered tiles on a four-by-four grid; equal tiles merge into one tile of double value, and the goal is to create a 2048 tile before the board fills up.

The program is split into four classes: `Game` (game loop and input), `GameBoard` (grid, movement and merging), `ScoreManager` (score) and `SaveManager` (file storage). It uses encapsulation, abstraction, constructors with initializer lists, object composition and output operator overloading, together with the C++ Standard Library for random numbers, formatting and file streams.

The game supports tile movement and merging, scoring, random tile generation, win and game-over detection, saving and loading, and restarting. This report describes the design, the way each OOP concept is used in the code, the algorithms, and the test results. An SFML graphical interface is listed as future work.

Keywords: C++, object-oriented programming, 2048, operator overloading, file handling, STL.

Contents

Abstract i

1 Introduction and Problem Statement 1

2 Objectives and Requirements 1

3 System Design 1

4 OOP Concepts in the Implementation 2

4.1 Encapsulation . . . 2
4.2 Abstraction . . . 2
4.3 Constructors and Initializer Lists . . . 3
4.4 Object Composition . . . 3
4.5 Operator Overloading . . . 3
4.6 Const-Correctness, STL and File Handling . . . 3
4.7 Error Handling . . . 4

5 Algorithms and Game Logic 4

6 Testing and Results 4

6.1 Output Screenshots . . . 5
6.2 Build and Execution . . . 6

7 Conclusion and Future Enhancements 6

A Source Code 8

A.1 main.cpp . . . 8
A.2 Game.cpp . . . 8
A.3 GameBoard.cpp . . . 10
A.4 ScoreManager.cpp . . . 14
A.5 SaveManager.cpp . . . 15
A.6 Game.h . . . 16
A.7 GameBoard.h . . . 16
A.8 ScoreManager.h . . . 16
A.9 SaveManager.h . . . 17

1 Introduction and Problem Statement

Object-oriented programming (OOP) builds software from classes that bundle data with the operations on it. MergeMind applies this to the 2048 puzzle: rather than writing the game in one large function, each responsibility (board, score, storage, game flow) is placed in its own class.

Problem statement. Implement the core rules of 2048: maintain a four-by-four grid, accept movement commands, merge equal tiles correctly, update the score, spawn new tiles, detect win and game-over, and allow the game to be saved and restored. Class declarations (`header/`) are kept separate from implementations (`.cpp` files).

Scope. The project is a terminal application. A graphical interface and animations are future enhancements.

2 Objectives and Requirements

Objectives:

1. Implement a playable 2048 game in C++.
2. Organize the program into separate, cooperating classes.
3. Apply encapsulation, abstraction and composition to game data and logic.
4. Display the board using output operator overloading.
5. Save and load games through file streams with validation.
6. Test normal gameplay and edge cases.

Table 1: Functional requirements

ID	Requirement
FR1	Initialize a four-by-four board with two starting tiles.
FR2	Move tiles in all four directions.
FR3	Merge equal tiles, at most once per tile per move.
FR4	Add the value of each merged tile to the score.
FR5	Spawn a new tile (2 or 4) only after a move that changes the board.
FR6	Detect the 2048 tile and the game-over condition.
FR7	Save and load the board and score, rejecting invalid data.
FR8	Restart the game at any time.
FR9	Display tiles in aligned columns in the terminal.

Non-functional requirements: readable and modular code that compiles with a C++17 compiler; an invalid move leaves the board unchanged; loaded data is validated before it replaces the current game.

3 System Design

Figure 1 shows the relationships. `Game` owns one object of each of the other three classes (filled diamond); `SaveManager` does not own the board or score but receives them as references when saving or loading (dashed arrow).

Project organization. Source files are in the project root and header files in `header/`; `main.cpp` only creates a `Game` object and calls `run()`.

```
MergeMind/
|-- header/  (Game.h, GameBoard.h, ScoreManager.h, SaveManager.h)
```

Table 2: Classes and their responsibilities

Class	Responsibility
`Game`	Runs the game loop, reads input and coordinates the other objects.
`GameBoard`	Stores the grid; implements movement, merging, tile spawning, win and game-over checks, and board display.
`ScoreManager`	Holds the score and updates it.
`SaveManager`	Writes and reads the save file.

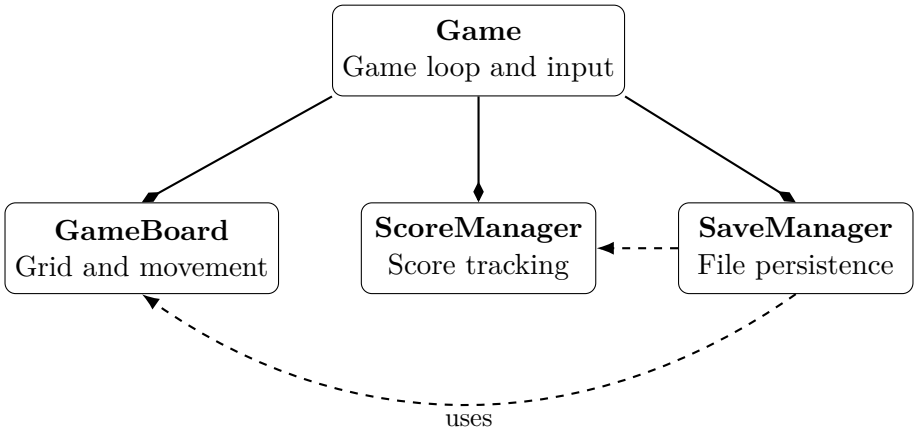


Figure 1: Class diagram of MergeMind

```
|-- Game.cpp, GameBoard.cpp, ScoreManager.cpp, SaveManager.cpp
|-- main.cpp
|-- README.md
```

4 OOP Concepts in the Implementation

4.1 Encapsulation

Each class keeps its data private and exposes only the operations other classes need. The grid in `GameBoard` and the score in `ScoreManager` cannot be modified from outside except through public methods that enforce the game rules:

```
class GameBoard {
private:
    static const int SIZE = 4;
    int board[SIZE][SIZE];            // hidden grid
    void addRandomTile();             // internal helpers
    bool moveLine(int index, bool isRow, bool towardStart, int& points);
public:
    MoveResult moveLeft();  MoveResult moveRight();
    MoveResult moveUp();    MoveResult moveDown();
    bool loadBoard(const int source[4][4]);   // validated write access
    ...
};
```

`Game` can never set a cell directly. Even the setters are guarded: `ScoreManager::addPoints` ignores non-positive values, `setScore` rejects negative values, and `loadBoard` rejects any board containing a negative value or a number that is not a power of two.

4.2 Abstraction

`Game::handleInput` only says *what* should happen (`gameBoard.moveLeft()`); it does not know how tiles are shifted. The four public move functions are thin wrappers over one private routine, `moveLine`,

which processes a single row or column and is parameterized by `isRow` and `towardStart`. Every move returns a small `MoveResult` structure (`changed`, `points`), so `Game` learns the outcome without seeing any board internals:

```
result = gameBoard.moveLeft();
...
if (result.changed) scoreManager.addPoints(result.points);
else cout << "The board did not change.\n";
```

4.3 Constructors and Initializer Lists

Every object starts in a valid state. `ScoreManager()` sets the score to zero, `GameBoard()` calls `reset()` which clears the grid and places two random tiles, and `Game()` initializes its members with an initializer list:

```
Game::Game()
    : gameBoard(), scoreManager(), saveManager(), wonMessageShown(false) {}
ScoreManager::ScoreManager() : score(0) {}
GameBoard::GameBoard() { reset(); }
```

4.4 Object Composition

`Game` *has* a `GameBoard`, a `ScoreManager` and a `SaveManager` as member objects, so their lifetimes are tied to the game and no manual memory management is needed. `Game` delegates work to them instead of doing it itself: movement to the board, scoring to the score manager, file I/O to the save manager.

```
class Game {
private:
    GameBoard gameBoard;
    ScoreManager scoreManager;
    SaveManager saveManager;
    bool wonMessageShown;
    ...
};
```

`SaveManager` works by association: `save` takes `const GameBoard&` and `const ScoreManager&`, while `load` takes non-const references so it can restore them.

4.5 Operator Overloading

The stream insertion operator is overloaded for `GameBoard` as a `friend` function, so the board can be printed with a single statement. It needs friend access to read the private grid, and it uses `std::setw(6)` to keep columns aligned for values up to four digits.

```
friend std::ostream& operator<<(std::ostream& out, const GameBoard& board);
...
cout << gameBoard;        // in Game::run()
```

4.6 Const-Correctness, STL and File Handling

Functions that do not modify an object are declared `const` (`hasWon`, `isGameOver`, `getScore`, `copyBoard`, `SaveManager::save`). Standard-library features used: `mt19937`, `random_device` and `uniform_int_distribution` for tile spawning (90% chance of 2, 10% chance of 4); `std::string`; `iomanip`; and `ofstream`/`ifstream` for saving and loading. File-scope constants and the random generator are placed in anonymous namespaces so they are private to their `.cpp` file.

The save file has a version header, the score, and four rows of tiles:

```
MERGEMIND_SAVE_V1
1632
32 128 64 16
4 64 16 8
4 8 4 0
2 0 0 0
```

4.7 Error Handling

The program checks stream and data validity and reports failures through return values; exceptions are not used. `SaveManager::save` returns false if the file cannot be opened or written. `SaveManager::load` returns false if the file is missing, the header is wrong, the score is negative, any value cannot be read, or `loadBoard` rejects the board. The board and score are only modified after *all* checks pass, so a corrupt file never leaves the game in an inconsistent state. `Game` then shows "Could not load the save file." Input failure (`cin`) ends the game loop cleanly.

Inheritance and polymorphism are not used, since the four classes have distinct roles and no "is-a" relationship; composition is the appropriate design here.

5 Algorithms and Game Logic

Movement and merging. For a move, `moveLine` reads a row or column in the direction of movement, removes zeros, merges equal neighbours, and writes the result back padded with zeros. After a merge the loop skips the next tile, so a newly merged tile cannot merge again in the same move. For example, moving [2, 0, 2, 4] left gives [4, 4, 0, 0]. The function compares the line before and after to report whether anything changed.

Score. Merging two tiles of value v creates a tile of value $2v$ and awards $2v$ points; for example, merging two 8s awards 16. The moves sum these points per line and return the total in `MoveResult`.

Random tile generation. `addRandomTile` collects all empty cells, picks one uniformly and places a 2 or 4. It is called only when `result.changed` is true, so invalid moves spawn nothing.

Win and game-over. `hasWon` is true when any tile is at least 2048; the congratulation message is shown once and play may continue. `isGameOver` is true only when there is no empty cell and no equal horizontal or vertical neighbours, so a full board with a possible merge is not game over.

6 Testing and Results

Table 3: Functional test cases and results

ID	Test	Expected result	Observed result	Status
T1	Initialization	Board starts with two valid tiles.	Two tiles (2/4) on an otherwise empty board (Fig. 2).	Pass
T2	Four directions	Tiles move in the chosen direction.	W, A, S, D shift tiles correctly.	Pass
T3	Merge rule	A tile merges at most once per move.	Equal pairs merge once per move.	Pass
T4	Score	Score rises by merged tile value.	Score rose from 1628 to 1632 after two 2s merged into a 4 (Fig. 3).	Pass
T5	Invalid move	No tile spawns after an unchanged move.	"The board did not change." and no new tile.	Pass
T6	Save/load	Board and score restored.	Game saved; file contents match board (Fig. 4).	Pass
T7	Restart	Board and score reset.	`N` starts a new game with score 0.	Pass
T8	End conditions	Win and game-over detected.	Messages shown at 2048 and on a stuck board.	Pass
T9	Display	Multi-digit tiles aligned.	Columns stay aligned up to 4 digits.	Pass

6.1 Output Screenshots

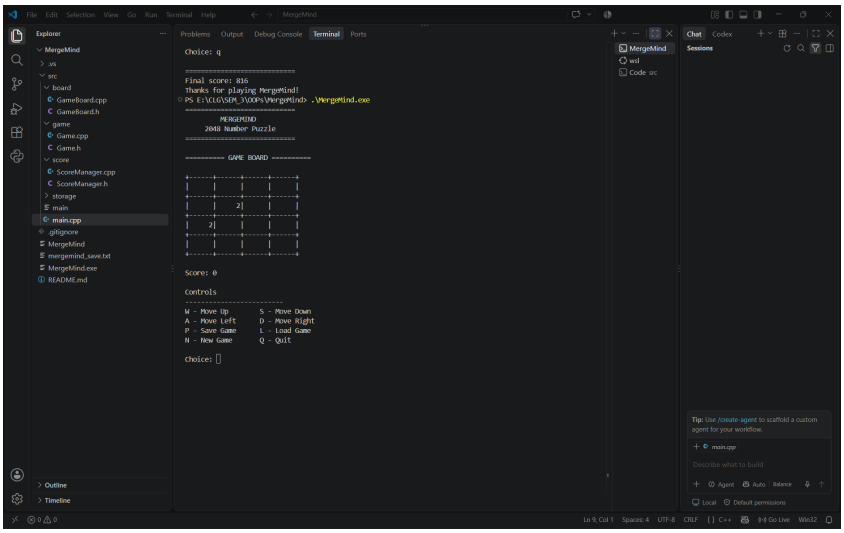


Figure 2: Initial board and game controls

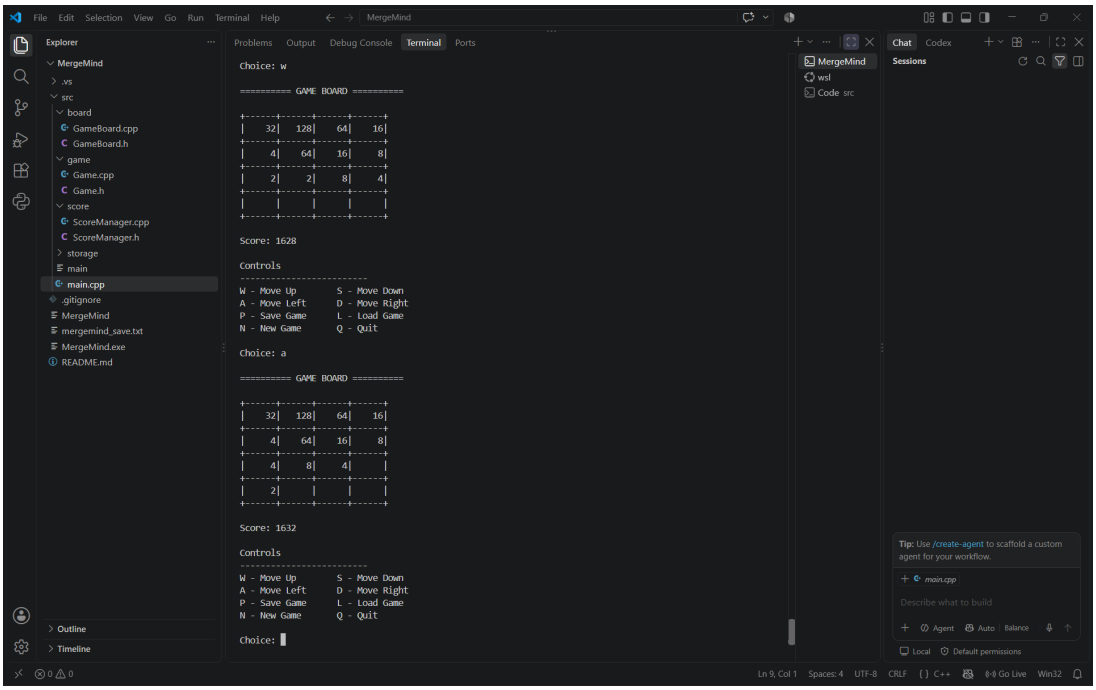


Figure 3: Tile merging and score tracking

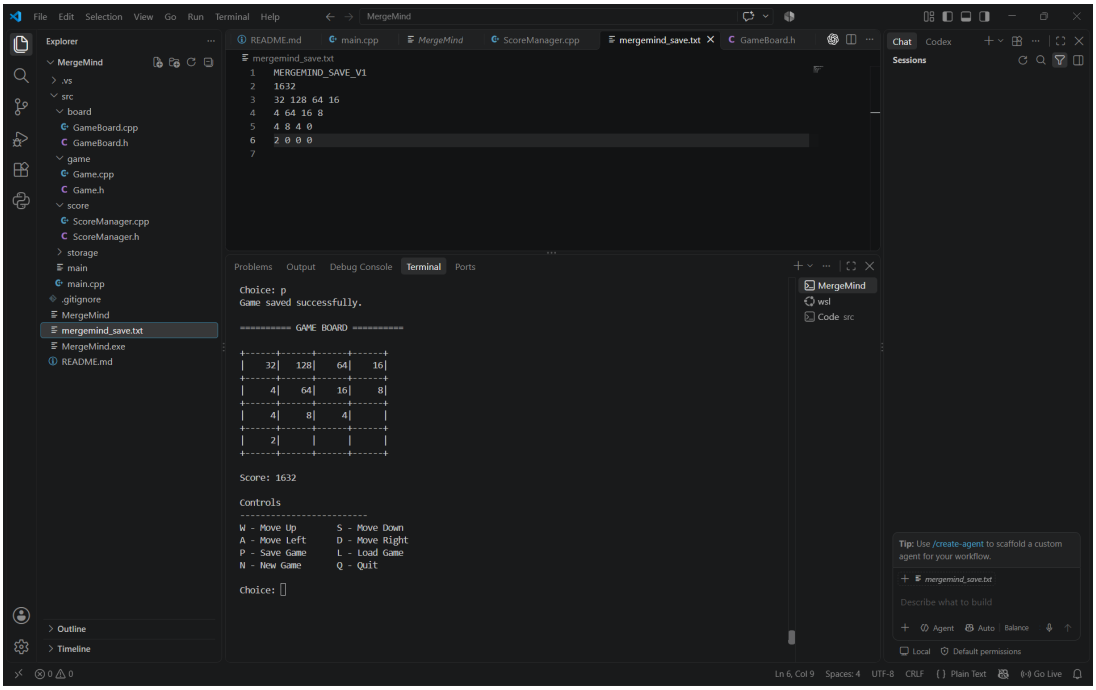


Figure 4: Save/load functionality

6.2 Build and Execution

The project requires GCC with C++17 support. From the project root:

```
g++ -std=c++17 GameBoard.cpp Game.cpp ScoreManager.cpp SaveManager.cpp main.cpp -o MergeMind
```

Run on Linux or WSL with `./MergeMind`, and on Windows (MinGW-w64) with `.\MergeMind.exe`.

7 Conclusion and Future Enhancements

MergeMind implements the main rules of 2048 using a clean class structure: `Game` coordinates, `GameBoard` handles board logic, `ScoreManager` tracks points and `SaveManager` handles files. Private data with validated public methods, a single reusable `moveLine` routine, composition of member objects, an overloaded « operator and validated file loading keep the code modular and robust.

Future work includes an SFML graphical interface, tile animations, an undo feature, improved input handling (no Enter key), and automated unit tests for movement, merging and save/load.

GitHub Repository: MergeMind – 2048 Number-Merging Puzzle Game

ANNEXURE

A Source Code

The following files contain the complete source code of MergeMind.

A.1 main.cpp

```cpp
#include "header/Game.h"

int main() {
    Game game;
    game.run();

    return 0;
}
```

A.2 Game.cpp

```cpp
#include <iostream>
#include <iomanip>
#include "header/Game.h"

using namespace std;

namespace {
    const string SAVE_FILE = "mergemind_save.txt";
}

Game::Game()
    : gameBoard(), scoreManager(), saveManager(),
      wonMessageShown(false) {
}

void Game::startNewGame() {
    gameBoard.reset();
    scoreManager.reset();
    wonMessageShown = false;

    cout << "\nNew game started!\n";
}

void Game::displayMenu() const {
    cout << "\nControls\n";
    cout << "-------------------------\n";
    cout << "W - Move Up        S - Move Down\n";
    cout << "A - Move Left      D - Move Right\n";
    cout << "P - Save Game      L - Load Game\n";
    cout << "N - New Game       Q - Quit\n";
    cout << "\n";
    cout << "Choice: ";
}

bool Game::handleInput(char choice) {
    MoveResult result = {false, 0};

    switch (choice) {
        case 'W':
        case 'w':
            result = gameBoard.moveUp();
            break;

        case 'A':
        case 'a':
            result = gameBoard.moveLeft();
            break;

        case 'S':
        case 's':
            result = gameBoard.moveDown();
            break;

```

```
        case 'D':
        case 'd':
            result = gameBoard.moveRight();
            break;

        case 'N':
        case 'n':
            startNewGame();
            return true;

        case 'P':
        case 'p':
            if (saveManager.save(
                    gameBoard, scoreManager, SAVE_FILE)) {
                cout << "Game saved successfully.\n";
            } else {
                cout << "Could not save the game.\n";
            }
            return true;

        case 'L':
        case 'l':
            if (saveManager.load(
                    gameBoard, scoreManager, SAVE_FILE)) {
                wonMessageShown = false;
                cout << "Game loaded successfully.\n";
            } else {
                cout << "Could not load the save file.\n";
            }
            return true;

        case 'Q':
        case 'q':
            return false;

        default:
            cout << "Invalid choice.\n";
            return true;
    }

    if (result.changed) {
        scoreManager.addPoints(result.points);
    } else {
        cout << "The board did not change.\n";
    }

    return true;
}

void Game::run() {
    cout << "============================\n";
    cout << "         MERGEMIND          \n";
    cout << "     2048 Number Puzzle     \n";
    cout << "============================\n";

    while (true) {
        cout << "\n========== GAME BOARD ==========\n";
        cout << gameBoard;
        cout << "\nScore: " << scoreManager.getScore() << '\n';

        if (gameBoard.hasWon() && !wonMessageShown) {
            cout << "\nCongratulations! You reached 2048!\n";
            wonMessageShown = true;
        }

        if (gameBoard.isGameOver()) {
            cout << "\n========== GAME OVER ==========\n";
            cout << "Final score: "
                 << scoreManager.getScore() << '\n';
            cout << "N - Start a new game\n";
            cout << "Q - Quit\n";
            cout << "Choice: ";

            char choice;

            if (!(cin >> choice)) {
```

```
                break;
            }

            if (choice == 'N' || choice == 'n') {
                startNewGame();
                continue;
            }

            if (choice == 'Q' || choice == 'q') {
                break;
            }

            cout << "Please enter N or Q.\n";
            continue;
        }

        displayMenu();

        char choice;

        if (!(cin >> choice)) {
            break;
        }

        if (!handleInput(choice)) {
            break;
        }
    }

    cout << "\n============================\n";
    cout << "Final score: " << scoreManager.getScore() << '\n';
    cout << "Thanks for playing MergeMind!\n";
}
```

A.3 GameBoard.cpp

```
#include <iostream>
#include <random>
#include <iomanip>
#include "header/GameBoard.h"

using namespace std;

namespace {
    mt19937 rng(random_device{}());
}

GameBoard::GameBoard() {
    reset();
}

void GameBoard::reset() {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            board[row][col] = 0;
        }
    }

    addRandomTile();
    addRandomTile();
}

void GameBoard::addRandomTile() {
    int emptyRows[SIZE * SIZE];
    int emptyCols[SIZE * SIZE];
    int emptyCount = 0;

    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            if (board[row][col] == 0) {
                emptyRows[emptyCount] = row;
                emptyCols[emptyCount] = col;
                emptyCount++;
```

```
            }
        }
    }

    if (emptyCount == 0) {
        return;
    }

    uniform_int_distribution<int> position(0, emptyCount - 1);
    uniform_int_distribution<int> tileChance(1, 10);

    int selected = position(rng);
    int row = emptyRows[selected];
    int col = emptyCols[selected];

    board[row][col] = (tileChance(rng) == 1) ? 4 : 2;
}

bool GameBoard::moveLine(int index, bool isRow,
                         bool towardStart, int& points) {
    int original[SIZE];
    int tiles[SIZE];
    int merged[SIZE];

    int count = 0;
    int outputCount = 0;
    points = 0;

    for (int i = 0; i < SIZE; i++) {
        int position = towardStart ? i : SIZE - 1 - i;

        int value = isRow
            ? board[index][position]
            : board[position][index];

        original[i] = value;

        if (value != 0) {
            tiles[count++] = value;
        }
    }

    for (int i = 0; i < count; i++) {
        if (i + 1 < count && tiles[i] == tiles[i + 1]) {
            merged[outputCount] = tiles[i] * 2;
            points += merged[outputCount];
            outputCount++;
            i++;
        } else {
            merged[outputCount++] = tiles[i];
        }
    }

    for (int i = 0; i < SIZE; i++) {
        int position = towardStart ? i : SIZE - 1 - i;

        if (isRow) {
            board[index][position] = 0;
        } else {
            board[position][index] = 0;
        }
    }

    for (int i = 0; i < outputCount; i++) {
        int position = towardStart ? i : SIZE - 1 - i;

        if (isRow) {
            board[index][position] = merged[i];
        } else {
            board[position][index] = merged[i];
        }
    }

    for (int i = 0; i < SIZE; i++) {
        int position = towardStart ? i : SIZE - 1 - i;

```

```
        int value = isRow
            ? board[index][position]
            : board[position][index];

        if (value != original[i]) {
            return true;
        }
    }

    return false;
}

MoveResult GameBoard::moveLeft() {
    MoveResult result = {false, 0};

    for (int row = 0; row < SIZE; row++) {
        int points = 0;

        if (moveLine(row, true, true, points)) {
            result.changed = true;
        }

        result.points += points;
    }

    if (result.changed) {
        addRandomTile();
    }

    return result;
}

MoveResult GameBoard::moveRight() {
    MoveResult result = {false, 0};

    for (int row = 0; row < SIZE; row++) {
        int points = 0;

        if (moveLine(row, true, false, points)) {
            result.changed = true;
        }

        result.points += points;
    }

    if (result.changed) {
        addRandomTile();
    }

    return result;
}

MoveResult GameBoard::moveUp() {
    MoveResult result = {false, 0};

    for (int col = 0; col < SIZE; col++) {
        int points = 0;

        if (moveLine(col, false, true, points)) {
            result.changed = true;
        }

        result.points += points;
    }

    if (result.changed) {
        addRandomTile();
    }

    return result;
}

MoveResult GameBoard::moveDown() {
    MoveResult result = {false, 0};

    for (int col = 0; col < SIZE; col++) {
```

```
        int points = 0;

        if (moveLine(col, false, false, points)) {
            result.changed = true;
        }

        result.points += points;
    }

    if (result.changed) {
        addRandomTile();
    }

    return result;
}

bool GameBoard::hasWon() const {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            if (board[row][col] >= 2048) {
                return true;
            }
        }
    }

    return false;
}

bool GameBoard::isGameOver() const {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            if (board[row][col] == 0) {
                return false;
            }
        }
    }

    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE - 1; col++) {
            if (board[row][col] == board[row][col + 1]) {
                return false;
            }
        }
    }

    for (int row = 0; row < SIZE - 1; row++) {
        for (int col = 0; col < SIZE; col++) {
            if (board[row][col] == board[row + 1][col]) {
                return false;
            }
        }
    }

    return true;
}

std::ostream& operator<<(std::ostream& out, const GameBoard& gameBoard) {
    const int cellWidth = 6;

    out << '\n';

    for (int row = 0; row < GameBoard::SIZE; row++) {
        out << "+------+------+------+------+\n";

        for (int col = 0; col < GameBoard::SIZE; col++) {
            out << "|";

            if (gameBoard.board[row][col] == 0) {
                out << std::setw(cellWidth) << "";
            } else {
                out << std::setw(cellWidth)
                    << gameBoard.board[row][col];
            }
        }

        out << "|\n";
```

```
    }

    out << "+------+------+------+------+\n";

    return out;
}


void GameBoard::copyBoard(int destination[4][4]) const {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            destination[row][col] = board[row][col];
        }
    }
}

bool GameBoard::loadBoard(const int source[4][4]) {
    // Validate the entire board before changing anything.
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            int value = source[row][col];

            if (value < 0) {
                return false;
            }

            if (value != 0 && (value < 2 || (value & (value - 1)) != 0)) {
                return false;
            }
        }
    }

    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            board[row][col] = source[row][col];
        }
    }

    return true;
}
```

A.4 ScoreManager.cpp

```
#include "header/ScoreManager.h"

ScoreManager::ScoreManager() : score(0) {
}

void ScoreManager::addPoints(int points) {
    if (points > 0) {
        score += points;
    }
}

int ScoreManager::getScore() const {
    return score;
}

void ScoreManager::reset() {
    score = 0;
}

bool ScoreManager::setScore(int value) {
    if (value < 0) {
        return false;
    }

    score = value;
    return true;
}
```

A.5 SaveManager.cpp

```cpp
#include <fstream>
#include "header/SaveManager.h"

using namespace std;

bool SaveManager::save(
    const GameBoard& gameBoard,
    const ScoreManager& scoreManager,
    const string& filename
) const {
    ofstream file(filename);

    if (!file.is_open()) {
        return false;
    }

    int board[4][4];
    gameBoard.copyBoard(board);

    file << "MERGEMIND_SAVE_V1\n";
    file << scoreManager.getScore() << '\n';

    for (int row = 0; row < 4; row++) {
        for (int col = 0; col < 4; col++) {
            file << board[row][col] << ' ';
        }
        file << '\n';
    }

    file.close();
    return !file.fail();
}

bool SaveManager::load(
    GameBoard& gameBoard,
    ScoreManager& scoreManager,
    const string& filename
) const {
    ifstream file(filename);

    if (!file.is_open()) {
        return false;
    }

    string header;
    int savedScore;
    int board[4][4];

    if (!(file >> header) || header != "MERGEMIND_SAVE_V1") {
        return false;
    }

    if (!(file >> savedScore) || savedScore < 0) {
        return false;
    }

    for (int row = 0; row < 4; row++) {
        for (int col = 0; col < 4; col++) {
            if (!(file >> board[row][col])) {
                return false;
            }
        }
    }

    // Validate before changing the current game.
    if (!gameBoard.loadBoard(board)) {
        return false;
    }

    scoreManager.setScore(savedScore);
    return true;
}
```

A.6 Game.h

```

#ifndef GAME_H
#define GAME_H

#include "GameBoard.h"
#include "ScoreManager.h"
#include "SaveManager.h"

class Game {
private:
    GameBoard gameBoard;
    ScoreManager scoreManager;
    SaveManager saveManager;
    bool wonMessageShown;

    void displayMenu() const;
    bool handleInput(char choice);
    void startNewGame();

public:
    Game();
    void run();
};

#endif
```

A.7 GameBoard.h

```
#ifndef GAMEBOARD_H
#define GAMEBOARD_H

#include <iostream>

struct MoveResult {
    bool changed;
    int points;
};

class GameBoard {
private:
    static const int SIZE = 4;
    int board[SIZE][SIZE];

    void addRandomTile();
    bool moveLine(int index, bool isRow, bool towardStart, int& points);

public:
    GameBoard();

    MoveResult moveLeft();
    MoveResult moveRight();
    MoveResult moveUp();
    MoveResult moveDown();

    bool hasWon() const;
    bool isGameOver() const;

    void copyBoard(int destination[4][4]) const;
    bool loadBoard(const int source[4][4]);

    void reset();

    friend std::ostream& operator<<(std::ostream& out, const GameBoard& board);
};

#endif
```

A.8 ScoreManager.h

```
#ifndef SCOREMANAGER_H
#define SCOREMANAGER_H

class ScoreManager {
private:
    int score;

public:
    ScoreManager();
    bool setScore(int value);
    void addPoints(int points);
    int getScore() const;
    void reset();
};

#endif
```

A.9 SaveManager.h

```

#ifndef SAVEMANAGER_H
#define SAVEMANAGER_H

#include <string>
#include "GameBoard.h"
#include "ScoreManager.h"

class SaveManager {
public:
    bool save(const GameBoard& gameBoard,
              const ScoreManager& scoreManager,
              const std::string& filename) const;

    bool load(GameBoard& gameBoard,
              ScoreManager& scoreManager,
              const std::string& filename) const;
};

#endif
```